

PinIT Career OS — Internship Program

Software Requirements Specification (SRS) and Step-by-Step Build Instructions

Field	Value
Document	PINIT_INTERNSHIP_SRS
Version	1.0
Date	30 September 2026
Written for	The Antigravity coding agent (and the owner, for review)
Written by	Claude Code, from the full project conversation and a scan of the codebase
Repository	Vinay-N-Kashyap/Present-Career-os
Working branch	<code>main-dis3ku</code> (merge to <code>main</code> only through a pull request)
Scope	Internships for the Python track (<code>python_ai</code>) of the 1, 3, 6, 9 and 12-month certification plans

Contents

PART A — READ THIS FIRST

- A1. How to use this document (for the Antigravity agent)
- A2. The standard checks (run after every task)
- A3. Mistakes the agent must avoid (seen before in this project)
- A4. How to merge to main

PART B — BACKGROUND: EVERYTHING DISCUSSED SO FAR

- B1. The product in one paragraph
- B2. What was finished in this conversation (Python track)
- B3. Owner decisions already made (do not change these)
- B4. What is still not built (honest status)
- B5. Internship code that already exists (found with the code graph)
- B6. Why other internship platforms fall short (the real-world problem)

PART C — REQUIREMENTS

- C1. Scope
- C2. The five internship tiers
- C3. Who can start which internship
- C4. Tier 1 — Python Job Simulation (1 month), detailed
- C5. Tier 2 — Virtual Internship - Backend (3 months), detailed

- C6. Tiers 3, 4 and 5 — real-world internships
- C7. Certificates
- C8. Switches and plan wording
- C9. Owner decisions needed (and the default used until then)
- C10. Data model (new tables)
- C11. API routes (all new, all server-checked)
- C12. AI generation and the validation pipeline (most important section)
- C13. Non-functional requirements
- C14. Acceptance criteria (the whole program)

PART D — STEP-BY-STEP TASKS FOR ANTIGRAVITY

- Phase 0 — Get ready
- Phase 1 — Honest plans and switches
- Phase 2 — Database
- Phase 3 — AI helper and validation
- Phase 4 — Tier 1: Python Job Simulation (backend)
- Phase 5 — Tier 1 (student screens)
- Phase 6 — Tier 2: Virtual Internship – Backend
- Phase 7 — Tiers 3–5 (build only after the owner decides D1 and D2)
- Phase 8 — Launch Tier 1 and Tier 2

PART E — REFERENCE

- E1. Glossary
- E2. Handy commands
- E3. Owner's short summary (non-technical)

PART A — READ THIS FIRST

A1. How to use this document (for the Antigravity agent)

This document is both a requirements specification and a work plan. Part D splits the whole job into **small numbered tasks** (T-01, T-02, ...). Follow these rules exactly:

1. **Do one task at a time, in order.** Do not start task N+1 until task N passes all its checks.
2. **Read every file listed under "Read first" before you change anything.** Most mistakes come from guessing what a file contains.
3. **Change only the files listed in the task.** If you think another file must change, stop and write a note to the owner instead.
4. **Run the "Verify" commands after every task.** If any check fails, fix it before moving on. Never skip, delete or weaken a test to make it pass.
5. **Commit after every task** with the message given in the task, on branch `main-dis3ku`, and push with `git push -u origin main-dis3ku`.
6. **Never invent facts.** Do not add fake companies, fake mentors, fake students, fake reviews, fake numbers, or features that are not built. If data is missing, show an honest empty state.
7. **Never flip** `INTERSHIP_AVAILABLE` (or the per-tier switches) to `true` except in the launch task that says so.
8. If a task is marked **BLOCKED** (needs owner decision), skip it and continue with the next unblocked task. Section C9 lists the decisions.
9. If you are unsure, **stop and ask the owner** rather than guessing. A wrong guess costs more than a question.

A2. The standard checks (run after every task)

```
npx tsc --noEmit      # must print nothing and exit 0
npm test              # every test must pass (162 at the time of writing, more as you
add tests)
npx next build         # must finish without errors
```

A task is **not done** until all three pass. Record the result in your commit message body (for example `tsc ok, npm test 170/170, build ok`).

A3. Mistakes the agent must avoid (seen before in this project)

#	Mistake	What to do instead
1	Declaring the same <code>const</code> twice in one scope	Use a different name, or reuse the first variable

#	Mistake	What to do instead
2	Duplicate keys in an inline style object (<code>left</code> twice)	One key per property; use a conditional value
3	Using <code>any</code> when a type is known	Import or declare the real type
4	Trusting data sent by the browser (progress, "passed", scores)	Only the server decides and records results
5	Sending hidden tests or reference solutions to the browser	Select explicit columns; never <code>select('*')</code> on task tables
6	Hand-writing projects or internship tasks into the code	Tasks are generated by AI at run time (owner rule)
7	Using AI output without checking it	Every generated task must pass the validation pipeline (Section B6)
8	Words the Python guard blocks inside tests or code (<code>requests</code> , <code>os.</code> , <code>sys.</code> , <code>open()</code> , <code>eval()</code> , <code>exec()</code> , <code>compile()</code> , <code>http.</code> , <code>socket</code> , <code>urllib</code> , <code>Path()</code> , <code>io.</code> , <code>subprocess</code>)	Rephrase; see <code>src/lib/code/python/pythonGuard.ts</code>
9	Promising things on plan cards that are not built	Plan text must match what is live (see T-03)
10	Editing the 24-month plan	The owner said: do not build the 24-month course . Leave <code>plan-24m-master</code> untouched
11	Big rewrites of working files	Make the smallest change that meets the task
12	Calling <code>/api/llm</code> from server code	Use the shared server helper built in T-08
13	Merging to <code>main</code> while "Web app tests" is red	Merge only when "Web app tests" passes (see A4)

A4. How to merge to `main`

- Open a pull request from `main-dis3ku` to `main`.
- The check **"Web app tests (database rules and API behaviour)"** must pass.
- The check **"Build, Test & Lint"** always fails today because it runs `pytest voice-service/tests`, and that folder does not exist. This failure is old and unrelated. Add one comment to the PR explaining this, and do not try to "fix" it inside an internship PR.
- Merge with the **merge** method (not squash, not rebase).

PART B — BACKGROUND: EVERYTHING DISCUSSED SO FAR

B1. The product in one paragraph

PinIT Career OS is a learning platform. Students buy a **certification plan** (1, 3, 6, 9, 12 or 24 months) on one of two tracks, **web_fullstack** or **python_ai**. Each month of a plan is one **course**. Each course has **30 days**; each day has a **long lesson (20-30 minutes)**, **two practice tasks** that are really checked, and a **test after every 5 lessons**. After the courses, the student builds a **capstone project** in 4 sprints and passes an **oral defense**, then gets a **verifiable certificate** with a QR code. The internship is the phase after the capstone. It has **not been built yet** and is hidden from students.

B2. What was finished in this conversation (Python track)

The 12-month Python plan is now **complete**: every month has its own course, with no repeats.

Month	Course id	Course
1	course-python-backend	Python Programming & Backend Systems
2	course-dsa-python	Data Structures & Algorithms in Python
3	course-database-eng	Database Engineering & Query Performance (SQL/PostgreSQL)
4	course-ai-python	AI Engineering in Python
5	course-distributed-python	Distributed Systems in Python
6	course-cloud-python	Cloud Engineering in Python (AWS)
7	course-nlp-python	NLP in Python
8	course-quant-python	Quantitative Trading Systems in Python
9	course-ai-prompt-python	Everyday AI & Prompt Engineering in Python
10	course-train-python	Distributed Model Training in Python (new, PR #4)
11	course-vector-python	Vector Search Engines in Python (new, PR #5)
12	course-safety-python	Production AI Safety & Guardrails in Python (new, PR #6)

Months 10, 11 and 12 used to repeat earlier courses (only 270 unique days instead of 360). Three new courses were built and merged. Each new course has:

- 30 days (`<name>PythonDays.ts`)
- 60 Python practice tasks, all checked by `npm test` against answers kept in `tests/fixtures/<name>_python_solutions.json` (answers are never shipped to students)
- 30 long lessons (`<name>PythonLongLessons.ts`), each about 9½–10½ minutes of spoken teaching and 21–22 minutes including code examples and quizzes. With the two practice tasks, each class is about **25-30 minutes**.

Plans that include these courses (Python track):

Plan	Courses
1 month (plan-1m-sprint)	Month 1
3 months (plan-3m-accelerator)	Months 1-3
6 months (plan-6m-pro)	Months 1-6
9 months (plan-9m-master)	Months 1-9
12 months (plan-12m-fellow)	Months 1-12

B3. Owner decisions already made (do not change these)

1. **Every lesson is a 20-30 minute class**, with real, checked tasks.
2. **Months 10-12** of the Python plan got 3 new courses (done).
3. **Do not build the 24-month course**. Leave the 24-month plan as it is.
4. **Projects must be generated by AI, not hand-written** into the code.
5. **The internship comes after the Python track is complete**. It is complete, so the internship is next.
6. The internship must **differ by plan length**: a 1-month student gets a different internship from a 3-month student, and so on.
7. Reports to the owner must be **short, simple and honest**.
8. Code goes to branch `main-dis3ku`; `main` is changed only through PRs that pass "Web app tests".

B4. What is still not built (honest status)

Area	Status
Teaching (lessons, practice, tests) for the 12-month Python plan	Done
Internship	Not built (this document)
Labs tab	Not built. Practice and tests happen inside lessons. Out of scope here
Per-course projects	Not built as hand-written content; owner wants AI-generated projects. The capstone exists
Web track courses, design-systems course	Later, out of scope here
24-month plan	Owner said do not build

B5. Internship code that already exists (found with the code graph)

The code graph (`graphify update src`, 5,901 nodes) and a text search found this existing work. **Reuse it; do not rebuild it.**

Piece	File	What it does today
Global switch	<code>src/lib/data/crashPlansData.ts</code> (<code>INTERNSHIP_AVAILABLE = false</code> , around line 80)	While false, <code>withoutInternship()</code> removes every internship promise from the plans shown to buyers
Plan field	<code>crashPlansData.ts</code> <code>internshipDurationMonths</code>	Says "2-3 Months" for the 1, 3, 6 and 9-month plans, "4-6 Months" for 12 months. Unrealistic; fixed in T-03
Progress phases	<code>src/lib/courses/crashCourseProgress.ts</code>	Phases: training → capstone → internship → graduation. Internship is complete when <code>enrollment.certificatesIssued.internshipCertHash</code> is set
Capstone (4 sprints)	<code>src/lib/courses/capstoneSprints.ts</code> , <code>src/app/api/quests/capstone/route.ts</code>	The server checks each sprint (GitHub links probed, live URL probed, signed defense result). This is the pattern to copy
Capstone desk UI	<code>src/app/quests/components/CapstoneInternshipPortal.tsx</code>	Shows sprints and issues the course certificate
Internship timeline UI	<code>src/app/quests/components/InternshipTimelineTracker.tsx</code> (used by <code>TrackSelectorDrawer.tsx</code>)	Timeline display
Internship log page	<code>src/app/internships/page.tsx</code>	Students record external internships (company, role, dates, mentor, skills)
Internship API	<code>src/app/api/internships/route.ts</code>	Reads and writes internship records; only the server can mark one verified
Database table	<code>internship_records</code> (created in <code>supabase/migrations/20260921_schema_mismatches_and_missing_tables.sql</code>)	<code>verified</code> , <code>verified_by</code> , <code>verified_at</code> are server-only
Career Intelligence tab	<code>src/app/career-intelligence/components/InternshipTrackerTab.tsx</code>	Shows internships and applications
Certificates	<code>src/app/api/certificates/course/route.ts</code> , <code>src/lib/certificates/courseCertificate.ts</code> , table <code>issued_certificates</code> , public page <code>src/app/verify/[credentialId]/page.tsx</code> , API <code>src/app/api/verify/[credentialId]/route.ts</code>	Issues and verifies QR certificates (prefix <code>PIN-CP-</code>)
Verification rules	<code>docs/PINIT_PROJECT_AND_INTERNSHIP_VERIFICATION_STANDARD.md</code> §4	A real internship needs an outside organisation, a named supervisor, at least 8 weeks full-time (≥ 30 h/week) or 16 weeks part-time (≥ 15 h/week), a supervisor evaluation, and evidence. In-house PinIT tasks do not count as an external internship

Piece	File	What it does today
AI calls	src/app/api/llm/route.ts	Uses OPENROUTER_API_KEY (model qwen/qwen-2.5-coder-32b-instruct) and GROQ_API_KEYS / GROQ_API_KEY (llama-3.3-70b-versatile)
AI project generator	src/app/api/projects/generate/route.ts, src/lib/projects/projectCatalog.ts	Generates project ideas with an LLM (has a curated fallback; the internship must not use hand-written fallbacks)
Python sandbox (server)	src/app/api/code/run-python/route.ts	Runs Python with python3, a 0.5–4 s timeout and a cleaned environment; blocks dangerous code with pythonGuard.ts
SQL engine	src/lib/code/sql/sqlCore.ts (runSqlPractice), package @electric-sql/pglite	Runs PostgreSQL in Node (already used by tests/sql_practice_tasks.test.ts)
Auth helpers	src/lib/server/requireAuth.ts (requireUserFromRequest, requireAdminUserFromRequest), src/lib/server/supabaseAdmin.ts, src/lib/server/rateLimit.ts, src/lib/server/publicUrlProbe.ts, src/lib/server/studentCourse.ts	Use these; never write new auth code
Old test script	scripts/tests/test_internships.cjs	Legacy script; not part of npm test

B6. Why other internship platforms fall short (the real-world problem)

Platform type	Examples	Problem
Job boards	Internshala, LinkedIn, Indeed	Only listings. Most students are never selected; no training, no feedback
Paid "certificate internships"	Many small sites	No real work, no checking. Employers do not trust the certificates
Virtual job simulations	Forage and similar	Good practice, but every student gets the same tasks, so answers are copied; little feedback
College-arranged internships	Campus programs	Quality varies; often clerical work; weak evidence

How PinIT will be better:

1. **Unique work per student:** every task is generated by AI for that student, so copying does not work.
2. **Everything is checked:** automatic hidden tests, then AI code review, then (from 3 months up) a human mentor.
3. **Honest names:** a simulation is called a simulation; only real outside work is called an industry internship.
4. **Verifiable proof:** every certificate has a QR code and a public page listing exactly what was done.

5. **Built on what the student learned:** tasks use only the skills of the courses in the student's plan.
-

PART C — REQUIREMENTS

C1. Scope

- **In scope:** the Python track (`python_ai`) of the 1m, 3m, 6m, 9m and 12m plans.
- **Out of scope:** the web track, the 24-month plan, the Labs tab, payments/pricing changes.

C2. The five internship tiers

Tier id	Plan	Name shown to students (default, see D3)	Kind	Length	Work	Human involved
t1_job_sim	1 month	Python Job Simulation	Solo, simulated company	2 weeks	5 AI-generated tickets in a pretend company's code: bug fixes, small features, data cleaning, refactoring. Checked by hidden tests + AI review	None needed
t2_virtual_team	3 months	Virtual Internship - Backend	Team of 3-4 (solo fallback), simulated company	4 weeks	AI-generated product backlog for a Python + SQL backend. Weekly sprints, pull requests, weekly checked tasks, async stand-ups, final demo	Mentor weekly review (D1); AI review until then
t3_project	6 months	Project Internship - AI Services	Real client or open source (D2); simulated client fallback	6-8 weeks	A deployed AI service for a real small client (NGO, startup) or an approved open-source project	Weekly mentor review (D1)
t4_industry	9 months	Verified Industry Internship	Real outside company (D2); open-source fallback	8-12 weeks (must meet the Verification Standard §4)	Work at a partner company, or ≥3 merged pull requests in approved open-source repos over ≥8 weeks	Company supervisor + PinIT verifier
t5_fellowship	12 months	Verified Fellowship + Placement	Real company role, paid where possible (D2)	3-6 months	Production AI system work; placement help (resume/ATS, mock interviews)	Company supervisor + PinIT verifier

Each tier is harder and more real than the one below: **solo simulation** → **team simulation** → **real client** → **real company** → **real job**.

C3. Who can start which internship

- **FR-ELIG-1:** A student may start the internship of **their own plan's tier only** (1m → t1_job_sim, 3m → t2_virtual_team, 6m → t3_project, 9m → t4_industry, 12m → t5_fellowship).
- **FR-ELIG-2:** The internship opens only when the student's **capstone is complete** (`isCapstoneComplete(enrollment)` in `crashCourseProgress.ts` returns true). The server checks this; the browser's view does not count.
- **FR-ELIG-3:** Only the `python_ai` track is eligible in this version.
- **FR-ELIG-4:** A tier whose switch is off (C8) cannot be started, even by calling the API directly.
- **FR-ELIG-5:** One active internship per student per crash enrollment.

C4. Tier 1 — Python Job Simulation (1 month), detailed

- **FR-T1-1:** On start, the server creates an internship enrollment with a **14-day deadline** (one extension of 7 days allowed, requested by the student, recorded by the server).
- **FR-T1-2:** The server generates a **pretend company profile** with AI (name, business, short README) that is unique per student. It must say clearly that it is fictional.
- **FR-T1-3:** The server generates **5 tickets** with AI, one of each type: (1) bug fix, (2) new function, (3) data cleaning, (4) refactor with no behaviour change, (5) small feature combining earlier work. Tickets use **only Month 1 Python skills** (functions, strings, lists, dicts, loops, errors, classes, simple modules, no files, no network).
- **FR-T1-4:** Tickets unlock **in order**. Ticket N+1 opens only after ticket N passes.
- **FR-T1-5:** Each ticket has a brief, starter code, **visible tests** (shown to the student) and **hidden tests** (never shown). A ticket **passes** only when the server runs the student's code against visible **and** hidden tests and all pass.
- **FR-T1-6:** After a pass, the server asks the AI for a **code review** (scores 1-5 for correctness, readability, edge cases and naming, plus 2 strengths and 2 improvements). The review is advice only; it never changes pass/fail.
- **FR-T1-7:** After all 5 tickets, the student writes a **final stand-up report** (100-300 words: what they built, one problem they solved, what they would improve). The server checks the length and asks the AI whether it matches the tickets (yes/no + reason). If no, the student can edit and resubmit.
- **FR-T1-8:** The server then issues the **internship certificate** (C7) and sets `certificatesIssued.internshipCertHash` on the crash enrollment.
- **FR-T1-9:** If the deadline passes (plus any extension), the status becomes `expired`. The student may restart **once** with newly generated tickets.

C5. Tier 2 — Virtual Internship - Backend (3 months), detailed

- **FR-T2-1:** On start, the student joins a **matching queue**. The server groups students who started within the same **7-day window** into teams of **3-4**. If no team forms in 7 days, the student continues in **solo mode** with the same backlog scaled to one person.
- **FR-T2-2:** The server generates with AI a **product brief** for a small Python + SQL backend (for example a library, clinic booking or canteen system), with a data model, 12-16 user

stories, and an assignment of stories to members. The skills used must come from Months 1–3 (Python, data structures and algorithms, SQL/PostgreSQL).

- **FR-T2-3:** The internship lasts **4 weekly sprints**. Each sprint has a goal and a due date.
- **FR-T2-4:** Every week, each member gets **2 AI-generated checked tasks** tied to their stories (one Python, one SQL). Python tasks are checked with the server Python sandbox; SQL tasks with PGlite (`runSqlPractice`). Same hidden-test rules as Tier 1.
- **FR-T2-5:** Each team has one **public GitHub repository** (link submitted by a team member, probed with `probePublicUrl`). Each member submits at least **one pull request link per sprint** (`https://github.com/<owner>/<repo>/pull/<n>` in the team repo). The server checks that the link exists and belongs to the team repo.
- **FR-T2-6:** Each member writes a **weekly async stand-up** (done / next / blockers, 30–200 words).
- **FR-T2-7:** At the end of each sprint, a **mentor review** (D1) approves or requests changes. **Until mentors exist**, the AI reviews the sprint (tasks passed, PR links, stand-ups) and an admin can spot-check. The AI review alone may approve Tier 2 sprints only while the D1 default applies (C9).
- **FR-T2-8: Final demo:** the team submits a deployed https URL (probed like capstone Sprint 3) and each member takes a short **AI oral defense** reusing the capstone interview signing (`evaluationSignature.ts` , `capstoneInterview.ts`) with the topic "Virtual Internship –".
- **FR-T2-9:** A member completes the internship when **all 4 sprints are approved, all their weekly tasks passed, and their defense passed**. The certificate then lists the project, the team size and the member's stories.

C6. Tiers 3, 4 and 5 — real-world internships

These tiers depend on real people and organisations, so most of their tasks are **BLOCKED until the owner decides D1 and D2**. The software parts that can be built without those decisions are marked as unblocked in Part D.

- **FR-T3-1 (Project Internship, 6m):** Admins post **opportunities** (real client projects from NGOs/startups, or approved open-source projects). Students apply; admins accept. Work lasts 6–8 weeks with weekly logs (hours, summary, links) and a weekly mentor review. The final evidence is a public repository and a deployed service (probed).
- **FR-T3-2 (fallback):** If no real opportunity is available when the student becomes eligible, they may take a **simulated-client project**: the AI plays a client in a chat with changing requirements. It must be named **"Project Internship (simulated client)"** on the certificate.
- **FR-T4-1 (Industry Internship, 9m):** Placement at a **partner company** that meets Verification Standard §4: named supervisor, ≥ 8 weeks full-time (≥ 30 h/week) or ≥ 16 weeks part-time (≥ 15 h/week), weekly logs, final **supervisor evaluation** (4 areas: technical competence & code quality, independent problem-solving, team communication & git collaboration, reliability). A PinIT admin verifies, then the server marks the linked `internship_records` row `verified = true`.
- **FR-T4-2 (fallback): Open-source track:** at least **3 merged pull requests** in approved repositories over at least **8 weeks**, checked with the public GitHub API (the PR is merged, the author matches the student's GitHub username, the dates fit).

- **FR-T5-1 (Fellowship, 12m):** A real company role for 3–6 months (paid where possible), verified like Tier 4, plus **placement support** using existing modules (resume/ATS in `src/lib/ats/`, interview practice in `src/app/interview/`). The supervisor may add a **recommendation letter**, stored as evidence.
- **FR-T345-1:** Supervisors submit evaluations through a **one-time secure link** sent by email (token stored only as a hash, expires in 14 days). Supervisors do not need a PinIT account.
- **FR-T345-2:** Organisations must pass the authenticity checks of Verification Standard §6 before their opportunities are shown (admin-only step).

C7. Certificates

- **FR-CERT-1:** Internship certificates use the existing `issued_certificates` table with a new kind `internship` and a new id prefix `PIN-IN-` (add to `src/lib/certificates/` next to `courseCertificate.ts`).
- **FR-CERT-2:** The public verify page (`/verify/[credentialId]`) must recognise `PIN-IN-` ids and show: student name, tier name, dates, what was done (tickets/stories/project), how it was checked (tests, AI review, mentor, supervisor), and whether it was **simulated** or **real**.
- **FR-CERT-3:** The certificate wording must be honest: Tiers 1–2 always say "simulation"/"virtual"; Tier 3 says "(simulated client)" when the fallback was used; only verified Tiers 4–5 say "Verified Industry Internship" / "Verified Fellowship".
- **FR-CERT-4:** Issuing the certificate sets `certificatesIssued.internshipCertHash` on the crash enrollment, so the existing progress code marks the internship phase complete.

C8. Switches and plan wording

- **FR-SW-1:** Replace the single switch with **one switch per tier** (`INTERNSHIP_TIER_AVAILABLE`), keeping `INTERNSHIP_AVAILABLE` as a derived value for old code (true if any tier is on). Every switch starts **false**.
- **FR-SW-2:** Plan cards and credential previews may mention an internship **only for tiers whose switch is on**, with that tier's real name and length.
- **FR-SW-3:** `internshipDurationMonths` must match C2: 1m "2 Weeks", 3m "4 Weeks", 6m "6-8 Weeks", 9m "8-12 Weeks", 12m "3-6 Months". Leave the 24-month plan unchanged.

C9. Owner decisions needed (and the default used until then)

Id	Question	Default until the owner answers	Tasks affected
D1	Will there be human mentors ? Who, and how many?	No mentors: AI review + admin spot check for Tier 2; Tiers 3–5 stay switched off	T-24, T-30...T-37
D2	Will the owner find partner companies / clients ?	None: Tiers 3–5 stay switched off; their software may be built but not launched	T-30...T-37
D3	May Tiers 1–2 be called " Job Simulation " and " Virtual Internship "?	Yes (recommended: honest and legally safer). Names live in one config file	T-05

Id	Question	Default until the owner answers	Tasks affected
D4	Is the internship included in the plan price?	Yes, included; no payment changes	none
D5	AI budget: which model and how many generations per student?	OpenRouter qwen/qwen-2.5-coder-32b-instruct ; at most 3 generation attempts per task; at most 2 full regenerations per student	T-08, T-10

C10. Data model (new tables)

Create one migration file `supabase/migrations/20261001_internship_program.sql`. Follow the rules in `supabase/migrations/README.md`: `CREATE TABLE IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`, idempotent, safe to re-run. **Enable RLS on every table and add no student policies:** all reads and writes go through server API routes using the admin client (`getSupabaseAdmin()`), like the capstone does.

Table	Key columns	Notes
internship_enrollments	id uuid pk, student_id uuid → auth.users, crash_enrollment_id uuid → user_crash_enrollments, tier text check in (t1_job_sim,t2_virtual_team,t3_project,t4_industry,t5_fellowship), track text default 'python_ai', status text check in (generating,active,completed,expired,withdrawn,generation_failed), started_at, due_at, extended bool default false, restarts int default 0, completed_at, certificate_id text, company_profile jsonb, final_report text, final_report_check jsonb, created_at, updated_at	Unique (student_id, crash_enrollment_id) where status in (generating, active)
internship_tasks	id, internship_enrollment_id, seq int, week int null, kind text, language text check in (python,sql), title, brief, starter_code, visible_tests, hidden_tests , reference_solution , sql_setup text null, skills jsonb, status text check in (locked,open,passed), attempts int default 0, passed_at, model text, generation_meta jsonb, created_at	hidden_tests and reference_solution must never leave the server
internship_submissions	id, task_id, student_id, code, passed bool, output text, ai_review jsonb null, created_at	output must never include hidden test source
internship_teams	id, tier, status (forming,active,completed), project_brief jsonb, repo_url text null, window_start date, created_at	Tier 2+
internship_team_members	team_id, student_id, internship_enrollment_id, stories jsonb, primary key (team_id, student_id)	
internship_sprints	id, team_id null, internship_enrollment_id null, number int, goal, due_at, status (open,submitted,approved,changes_requested), review jsonb, reviewed_by uuid null, reviewed_at	Exactly one of team or enrollment is set
internship_pr_links	id, sprint_id, student_id, url, checked_at	

Table	Key columns	Notes
internship_standups	id, internship_enrollment_id, week int, done text, next text, blockers text, created_at	
internship_opportunities	id, org_name, org_website, kind (client_project, open_source, industry, fellowship), title, description, min_tier, seats int, paid bool, stipend numeric, authenticity_tier text, status (draft, open, closed), created_by uuid, created_at	Admin-only writes
internship_applications	id, opportunity_id, student_id, internship_enrollment_id, status (applied, shortlisted, accepted, rejected, withdrawn), created_at	
internship_weekly_logs	id, internship_enrollment_id, week, hours numeric, summary, links jsonb, created_at	Tiers 3-5
internship_supervisor_evaluations	id, internship_enrollment_id, supervisor_name, supervisor_email, token_hash, token_expires_at, ratings jsonb, comments, submitted_at	Token stored only as SHA-256 hash

For Tiers 4-5, the verified outcome is also written to the existing `internship_records` table (server only), so the existing Internship page and Career Intelligence tab show it.

C11. API routes (all new, all server-checked)

Every route: `requireUserFromRequest` (or `requireAdminUserFromRequest` for admin routes), `checkRateLimit`, input validated with `zod` (see `src/lib/server/validate.ts`), errors returned as `{ ok: false, error: 'CODE', message: 'plain words' }` like `src/app/api/quests/capstone/route.ts`.

Method & path	Who	Purpose
GET <code>/api/internship</code>	student	The student's internship: tier, status, deadline, tasks without hidden fields , sprints, team
POST <code>/api/internship/start</code>	student	Checks eligibility (C3), creates the enrollment, starts generation
POST <code>/api/internship/extend</code>	student	One 7-day extension (Tier 1)
POST <code>/api/internship/restart</code>	student	Restart once after expired
POST <code>/api/internship/tasks/[taskId]/submit</code>	student	Runs visible + hidden tests on the server; records the submission; unlocks the next task; triggers the AI review
POST <code>/api/internship/report</code>	student	Final stand-up report (Tier 1)
POST <code>/api/internship/standup</code>	student	Weekly stand-up (Tier 2+)

Method & path	Who	Purpose
POST /api/internship/sprint/[sprintId]/pr	student	Adds a PR link (checked)
POST /api/internship/sprint/[sprintId]/submit	student	Marks the sprint ready for review
POST /api/internship/team/repo	student	Sets the team repository (checked)
POST /api/internship/demo	student	Final demo URL + signed defense result
POST /api/internship/certificate	student	Issues the certificate when everything is complete
POST /api/admin/internship/sprint/[sprintId]/review	admin (later mentor)	Approve or request changes
GET/POST /api/admin/internship/opportunities	admin	Manage opportunities (Tiers 3-5)
POST /api/internship/opportunities/[id]/apply	student	Apply (Tiers 3-5)
POST /api/internship/weekly-log	student	Weekly log (Tiers 3-5)
POST /api/admin/internship/[id]/supervisor-link	admin	Creates the one-time supervisor link
GET/POST /api/internship/supervisor/[token]	supervisor (no login)	Shows the form / submits the evaluation
POST /api/admin/internship/[id]/verify	admin	Marks a Tier 4-5 internship verified

C12. AI generation and the validation pipeline (most important section)

AI makes mistakes. While building the Month 10-12 courses, more than **20 expected answers in AI-written tasks were wrong** and had to be fixed by running the code. Therefore **no AI-generated task may reach a student until the server has proved it works**.

Generation (per task):

1. Build the prompt from: the tier, the skill list of the student's completed courses, the task kind, the company profile, and a random **seed** (stored in `generation_meta`).
2. Ask the model for **strict JSON** with fields `title`, `brief`, `starter_code`, `visible_tests`, `hidden_tests`, `reference_solution`, `skills` (and `sql_setup` for SQL tasks).

3. Parse the JSON with a `zod` schema. On failure, retry (max 3 attempts per task, D5).

Validation (all must pass, in this order):

Step	Check	Why
V1	Every text field is within length limits (brief $\leq$ 2,000 chars, code $\leq$ 6,000, tests $\leq$ 6,000)	Stops runaway output
V2	<code>findForbiddenPython</code> (from <code>pythonGuard.ts</code>) finds nothing in starter code, tests or solution	The sandbox refuses those patterns
V3	<code>reference_solution</code> + <code>visible_tests</code> pass in the sandbox	The task is solvable
V4	<code>reference_solution</code> + <code>hidden_tests</code> pass in the sandbox	The hidden tests are correct
V5	<code>starter_code</code> + <code>visible_tests</code> + <code>hidden_tests</code> fail	The task is not already solved
V6	The brief does not contain the reference solution (no line of the solution longer than 20 characters appears in the brief)	No answer leaks
V7	Hidden tests contain at least 3 <code>assert</code> lines, visible tests at least 2	Enough checking

If all 3 attempts fail, set the enrollment to `generation_failed` and show: "We could not prepare your internship right now. Please try again in a few minutes." **Never** fall back to hand-written tasks (owner rule B3-4).

AI review and report checks use the same helper with a strict JSON schema. If the AI call fails, the student's pass still stands and the review shows "Review not available right now."

Cost control: rate-limit generation (1 start per student per 10 minutes), store every generation, never regenerate a task that already passed.

C13. Non-functional requirements

Id	Requirement
NFR-SEC-1	Hidden tests and reference solutions are never sent to the browser. A test must prove it (T-12)
NFR-SEC-2	Only the server writes progress, passes, reviews, approvals and certificates
NFR-SEC-3	All new tables have RLS enabled and no student policies
NFR-SEC-4	Supervisor tokens are random (32 bytes), stored as SHA-256 hashes, single-use, and expire in 14 days
NFR-SEC-5	Student code runs only in the existing sandbox with its timeout and guard
NFR-PRIV-1	Show supervisors only the student's name, tier, dates and logged work, never other personal data
NFR-HON-1	No fake companies, mentors, reviews, numbers or testimonials anywhere. Empty states must be honest

Id	Requirement
NFR-HON-2	Simulated work is always labelled as simulated (C7)
NFR-PERF-1	Generating 5 Tier-1 tasks finishes within 2 minutes; the UI shows progress while generating
NFR-UX-1	Plain, simple English in every student-facing message
NFR-UX-2	Mobile-friendly layouts; respect the sidebar rules in <code>CLAUDE.md</code>
NFR-TEST-1	Every new server module has tests in <code>tests/*.test.ts</code> (run by <code>npm test</code>); AI calls are replaced by a fake in tests

C14. Acceptance criteria (the whole program)

1. A 1-month Python student who finished the capstone can start the Job Simulation, receives 5 unique checked tickets, passes them, writes the report and gets a `PIN-IN-` certificate that verifies publicly as "Python Job Simulation (simulated company)".
2. A 3-month student can join or form a team, complete 4 sprints with checked tasks, PR links, stand-ups and reviews, pass the demo and defense, and get "Virtual Internship - Backend".
3. No API response ever contains `hidden_tests` or `reference_solution`.
4. Plans show internship promises only for tiers that are switched on, with the correct names and lengths.
5. The 24-month plan is unchanged.
6. `npx tsc --noEmit`, `npm test` and `npx next build` all pass.

PART D — STEP-BY-STEP TASKS FOR ANTIGRAVITY

Each task is small on purpose. **Do them in order.** Each task lists: goal, files to read first, files to change, steps, what not to do, how to verify, and when it is done.

Phase 0 — Get ready

T-01 — Read and confirm the starting point

- **Goal:** Understand the existing code before changing anything.
- **Read first:** this whole document; `CLAUDE.md`;
`docs/PINIT_PROJECT_AND_INTERNSHIP_VERIFICATION_STANDARD.md` §4 and §6;
`src/lib/data/crashPlansData.ts`; `src/lib/courses/crashCourseProgress.ts`;
`src/lib/courses/capstoneSprints.ts`; `src/app/api/quests/capstone/route.ts`;
`src/app/api/certificates/course/route.ts`.
- **Steps:** 1. `git checkout main-dis3ku && git pull origin main-dis3ku && git merge origin/main` 2. Run the three standard checks (A2). Write down the test count.
- **Do not:** change any file.
- **Done when:** all three checks pass and you can explain in 5 lines how the capstone route checks a sprint.

T-02 — Create the internship folder and a README

- **Goal:** A home for internship code.
- **Files to create:** `src/lib/internships/README.md`
- **Steps:** Write 10–20 lines: what the folder is for, a link to this SRS (`docs/internship-srs/PINIT_INTERNSHIP_SRS.md`), and the rule "hidden tests and reference solutions never leave the server".
- **Verify:** standard checks.
- **Commit:** Add `internships` folder README
- **Done when:** the file exists and the checks pass.

Phase 1 — Honest plans and switches

T-03 — Fix internship lengths in the plans

- **Goal:** Plan data matches C2 lengths.
- **Read first:** `src/lib/data/crashPlansData.ts` (every `internshipDurationMonths`).
- **Files to change:** `src/lib/data/crashPlansData.ts` only.
- **Steps:** Set `internshipDurationMonths` to `'2 Weeks'` (1m), `'4 Weeks'` (3m), `'6-8 Weeks'` (6m), `'8-12 Weeks'` (9m), `'3-6 Months'` (12m). In the `features` and `journeySteps` of those

five plans, change internship lengths in the text to the same values ("2-3 Months Real-Time Industry Internship" becomes, for example, "2-Week Python Job Simulation" for 1m). Use the tier names from C2.

- **Do not:** touch `plan-24m-master`. Do not change prices. Do not change `INTERNSHIP_AVAILABLE`.
- **Verify:** standard checks; also `grep -n "internshipDurationMonths" src/lib/data/crashPlansData.ts` shows the new values.
- **Commit:** Make internship lengths in plans match the internship tiers
- **Done when:** the values match and the checks pass. (Students still see no internship text because the switch is off.)

T-04 — Per-tier switches

- **Goal:** One switch per tier (FR-SW-1).
- **Files to change:** `src/lib/data/crashPlansData.ts`.
- **Steps:** 1. Add `export type InternshipTier = 't1_job_sim' | 't2_virtual_team' | 't3_project' | 't4_industry' | 't5_fellowship';` 2. Add `export const PLAN_TIER_TO_INTERNSHIP: Record<string, InternshipTier> mapping '1m', '3m', '6m', '9m', '12m' to the tiers (no '24m')`. 3. Add `export const INTERNSHIP_TIER_AVAILABLE: Record<InternshipTier, boolean>` with every value `false`. 4. Change `INTERNSHIP_AVAILABLE` to `export const INTERNSHIP_AVAILABLE = Object.values(INTERNSHIP_TIER_AVAILABLE).some(Boolean);` 5. Change the plan filter so `withoutInternship(plan)` is applied to each plan **unless** that plan's tier switch is on.
- **Do not:** set any switch to true.
- **Verify:** standard checks; add a test in `tests/internship_switches.test.ts`: with all switches false, no plan in `CRASH_COURSE_PLANS` has an internship feature (regex `/intern|fellowship|apprentice/i`) and `INTERNSHIP_AVAILABLE` is false.
- **Commit:** Add per-tier internship switches (all off)
- **Done when:** the new test passes along with all old tests.

T-05 — Tier configuration file

- **Goal:** One file holds every tier's name, length, rules and skills (so names can change in one place, D3).
- **Files to create:** `src/lib/internships/tiers.ts`
- **Steps:** Export `INTERNSHIP_TIERS: Record<InternshipTier, TierConfig>` where `TierConfig` has: `name`, `certificateName`, `simulated: boolean`, `lengthDays` (14, 28, 56, 84, 180), `taskCount` (5 for t1, 8 per member for t2 = 2 per week × 4, 0 for t3–t5), `teamSize` ([1,1] for t1, [3,4] for t2, [1,1] otherwise), `courseIds` (the plan's Python course ids from B2), `skills` (short plain-English skill list per course), `needsMentor`, `needsPartner`. Use the values in C2.
- **Verify:** standard checks + a test that every tier has a config and that `courseIds` match the plan modules in `crashPlansData.ts` for `python_ai`.
- **Commit:** Add internship tier configuration

Phase 2 — Database

T-06 — Migration: internship tables

- **Goal:** Create the tables in C10.
- **Read first:** `supabase/migrations/README.md`;
`supabase/migrations/20260924_user_crash_enrollments_and_wallet_schema.sql` (style);
`supabase/migrations/20260921_schema_mismatches_and_missing_tables.sql` lines around
`internship_records` (style for RLS).
- **Files to create:** `supabase/migrations/20261001_internship_program.sql`. **Files to change:**
`supabase/migrations/README.md` (add the file to the ordered list).
- **Steps:** Write every table from C10 with `CREATE TABLE IF NOT EXISTS`, the `check`
constraints, foreign keys with `ON DELETE CASCADE` for student-owned rows, indexes on
`student_id`, `internship_enrollment_id`, `team_id`, and `ALTER TABLE ... ENABLE ROW LEVEL`
`SECURITY`; for each table. No student policies.
- **Do not:** edit old migration files or `PRODUCTION_CONSOLIDATED_MIGRATIONS.sql`.
- **Verify:** standard checks; if `tests/db/migration-chain.test.ts` checks the file list, update it
as that test expects (read it first).
- **Commit:** Add internship program tables
- **Done when:** the SQL runs twice in a row on a fresh database without errors (the owner
or CI runs it; you can test locally with PGlite if you like).

T-07 — TypeScript row types

- **Files to create:** `src/lib/internships/types.ts`
- **Steps:** Declare interfaces for every table row (`snake_case`, as stored) and the client-safe
shapes (`camelCase`). The client-safe task type **must not have** `hiddenTests` or
`referenceSolution` fields.
- **Verify:** standard checks.
- **Commit:** Add internship row types

Phase 3 — AI helper and validation

T-08 — Server helper for strict-JSON AI calls

- **Goal:** One safe way for server code to ask the AI for JSON.
- **Read first:** `src/app/api/llm/route.ts` (how OpenRouter/Groq keys and requests work).
- **Files to create:** `src/lib/server/llmJson.ts`
- **Steps:** Export `askForJson<T>(opts: { system: string; user: string; schema: ZodSchema<T>;
maxTokens: number; model?: string }): Promise<{ ok: true; data: T; model: string } | {
ok: false; reason: string }>`. Use `OPENROUTER_API_KEY` first (default model from D5), then
Groq keys as fallback. Extract the JSON from the reply (first `{` to last `}`), parse it, and
validate with the schema. Timeout 60 s. Export a way for tests to inject a fake (for
example `setLlmJsonTransportForTests(fn)`).
- **Do not:** call `/api/llm` over HTTP; log API keys; use `any`.
- **Verify:** standard checks + `tests/internship_llm_json.test.ts` using the fake: valid JSON
passes, chatter around JSON is handled, invalid JSON returns `ok: false`, schema
mismatch returns `ok: false`.

- **Commit:** Add server helper for strict-JSON AI calls

T-09 — Extract the Python sandbox into a reusable function

- **Goal:** Run student code + tests on the server from internship code, without HTTP.
- **Read first:** `src/app/api/code/run-python/route.ts` (the part that writes a temp file and runs `python3` with a timeout and a clean environment).
- **Files to create:** `src/lib/server/pythonSandbox.ts`. **Files to change:** `src/app/api/code/run-python/route.ts` (use the new function; **behaviour must not change**).
- **Steps:** Move the temp-file + `exec` + timeout + clean-env logic into `runPythonInSandbox({ code, tests, timeoutMs })`: `Promise<{ passed: boolean; stdout: string; stderr: string; timedOut: boolean }>`. Keep the guard check (`findForbiddenPython`) inside it.
- **Do not:** change the route's request or response shape, its rate limits or its pass-receipt logic.
- **Verify:** standard checks + `tests/internship_python_sandbox.test.ts` (skip the test with a clear message if `python3` is not installed): passing code passes, failing assert fails, an infinite loop times out, `import os` is refused.
- **Commit:** Extract the Python sandbox into a shared server function

T-10 — Task generation schema and prompt builder

- **Files to create:** `src/lib/internships/generateTask.ts`
- **Steps:** Export a `zod` schema for a generated task (fields in C12), and `buildTaskPrompt({ tier, kind, skills, companyProfile, seed, language })` returning `{ system, user }`. The system prompt must require: only the listed skills; no files, network, `input()` or randomness; plain-English brief; starter code that runs but fails the tests; tests as plain `assert` lines; no forbidden words (list them).
- **Verify:** standard checks + a test that the prompt contains every skill and the forbidden-word list.
- **Commit:** Add internship task schema and prompt builder

T-11 — The validation pipeline (V1-V7)

- **Files to create:** `src/lib/internships/validateTask.ts`
- **Steps:** Export `validateGeneratedTask(task): Promise<{ ok: true } | { ok: false; step: 'V1'|'V2'|...; reason: string }>` implementing C12 V1-V7 in order. Python uses `runPythonInSandbox`; SQL uses `runSqlPractice` from `src/lib/code/sql/sqlCore.ts` with `sql_setup`.
- **Verify:** standard checks + `tests/internship_validate_task.test.ts` with hand-made **test fixtures** (fixtures are allowed in tests; they are not student content): a good task passes; a wrong hidden test fails at V4; an already-solved starter fails at V5; a forbidden word fails at V2; a leaked solution fails at V6.
- **Commit:** Add validation pipeline for AI-generated internship tasks

T-12 — Generate-and-validate loop + "no hidden data leaves the server" guard

- **Files to change:** `src/lib/internships/generateTask.ts`. **Files to create:** `src/lib/internships/toClient.ts`
- **Steps:** 1. `generateValidatedTask(input)`: up to 3 attempts of ask → validate; return the first valid task or `{ ok: false, reasons }`. 2. `taskToClient(row)`: returns only `id, seq, week, kind, language, title, brief, starterCode, visibleTests, skills, status, attempts, passedAt`.
- **Verify:** standard checks + tests: with a fake AI returning bad then good output, the loop returns the good one on attempt 2; `taskToClient` output has **no** `hidden_tests`, `hiddenTests`, `reference_solution` or `referenceSolution` key (check with `Object.keys`).
- **Commit:** Add generate-and-validate loop and client-safe task shape

Phase 4 — Tier 1: Python Job Simulation (backend)

T-13 — Eligibility check

- **Files to create:** `src/lib/internships/eligibility.ts`
- **Steps:** `checkInternshipEligibility({ enrollment, plan, track })` returns `{ ok: true, tier }` or `{ ok: false, error, message }` covering C3 (own tier only, capstone complete via `isCapstoneComplete`, `python_ai` only, tier switch on, no active internship).
- **Verify:** standard checks + tests for each failure reason.
- **Commit:** Add internship eligibility check

T-14 — Company profile generation

- **Files to create:** `src/lib/internships/companyProfile.ts`
- **Steps:** `generateCompanyProfile({ tier, seed })` via `askForJson` with schema `{ name, industry, readme }`. The README must start with "This is a fictional company created for your internship simulation." Reject (retry) if missing.
- **Verify:** tests with the fake AI.
- **Commit:** Add fictional company profile generation

T-15 — POST `/api/internship/start` and GET `/api/internship`

- **Read first:** `src/app/api/quests/capstone/route.ts`, `src/lib/server/studentCourse.ts`.
- **Files to create:** `src/app/api/internship/route.ts` (GET), `src/app/api/internship/start/route.ts` (POST).
- **Steps:** Start: auth, rate limit (1 per 10 min), load the crash enrollment with `loadStudentCourse`, eligibility, insert `internship_enrollments` (`status: generating`), generate the company profile and the 5 Tier-1 tasks (kinds from FR-T1-3; only ticket 1 open, others locked), set `status: active`, `due_at = now + 14 days`. If generation fails: `status: generation_failed`. GET: return the enrollment and tasks via `taskToClient`.
- **Do not:** return hidden fields; select `*` from `internship_tasks`.

- **Verify:** standard checks + route tests with a fake database/AI if the project has a pattern (read `tests/test_pass_receipts.test.ts` for how server code is tested); at minimum, unit-test the pure functions the routes call.
- **Commit:** Add internship start and read routes

T-16 — Submit a ticket

- **Files to create:** `src/app/api/internship/tasks/[taskId]/submit/route.ts`
- **Steps:** Auth, rate limit (10 per minute), load the task (must belong to the student, be `open`, enrollment `active` and not past `due_at`), run code with visible + hidden tests, store `internship_submissions` (output trimmed to 4,000 chars, with hidden test **source** never included; show only "Hidden check N failed"), increment `attempts`; on pass set `passed`, open the next task, and return `{ ok: true, passed: true }`.
- **Verify:** standard checks + tests of the pass/fail/locked/expired paths using pure helpers.
- **Commit:** Add internship ticket submission with hidden tests

T-17 — AI code review after a pass

- **Files to create:** `src/lib/internships/codeReview.ts`; change the submit route to call it after a pass (do not block the response for more than 20 s; if it fails, store `ai_review: null`).
- **Steps:** Schema `{ scores: { correctness, readability, edgeCases, naming } (1-5), strengths: string[2], improvements: string[2] }`. The prompt must not include hidden tests or the reference solution.
- **Commit:** Add AI code review for passed internship tickets

T-18 — Final report, extension, restart

- **Files to create:** `src/app/api/internship/report/route.ts`, `extend/route.ts`, `restart/route.ts`.
- **Steps:** Report: only after all 5 tickets pass; 100–300 words; AI relevance check `{ matches: boolean, reason }`; store. Extend: once, +7 days, only while active. Restart: once, only when `expired`; generates new tickets.
- **Commit:** Add internship report, extension and restart

T-19 — Internship certificate

- **Read first:** `src/lib/certificates/courseCertificate.ts`, `src/app/api/certificates/course/route.ts`, `src/app/api/verify/[credentialId]/route.ts`, `src/app/verify/[credentialId]/page.tsx`.
- **Files to create:** `src/lib/certificates/internshipCertificate.ts` (prefix `PIN-IN-`, kind `internship`), `src/app/api/internship/certificate/route.ts`. **Files to change:** the verify API route and page to recognise `PIN-IN-`.
- **Steps:** Issue only when complete (Tier 1: 5 passed + report accepted). Insert into `issued_certificates` (same signing as course certificates). Set

`certificatesIssued.internshipCertHash` on the crash enrollment. The verify page shows the fields in FR-CERT-2 and the word "simulated" for Tiers 1-2.

- **Verify:** standard checks + tests for the id prefix and the "complete" rule.
- **Commit:** Add internship certificates and verification

Phase 5 — Tier 1 (student screens)

T-20 — Internship panel in the course view

- **Read first:** `CapstoneInternshipPortal.tsx`, `VerticalCheckpointStepper.tsx`, `InternshipTimelineTracker.tsx`, `TrackSelectorDrawer.tsx`.
- **Files to create:** `src/app/quests/components/internship/InternshipPanel.tsx` (+ small child components in the same folder, each under 300 lines).
- **Steps:** States: not eligible (say why, in plain words), ready to start (Start button → `POST /start`), generating (progress text), generation failed (retry later), active (company card, deadline, ticket list with lock/open/passed), expired (restart button if allowed), completed (certificate link).
- **Do not:** compute pass/fail in the browser; show fake data.
- **Commit:** Add internship panel for students

T-21 — Ticket workspace

- **Files to create:** `src/app/quests/components/internship/TicketWorkspace.tsx`
- **Steps:** Show brief, starter code in the existing code editor component used by lessons (find it via `src/app/quests/lesson/`), visible tests, a "Run visible tests" button (browser Pyodide is fine for visible tests only), and a "Submit" button that calls the server. Show server results and the AI review.
- **Commit:** Add internship ticket workspace

T-22 — Report and certificate screens

- **Steps:** Report form with a live word count; certificate card with QR (reuse `qrcode.react` as in `CapstoneInternshipPortal.tsx`).
- **Commit:** Add internship report and certificate screens

T-23 — Show the internship phase in progress views

- **Files to change:** `src/lib/courses/crashCourseProgress.ts` (use the per-tier switch for the enrollment's plan tier instead of the global one), and the components that show phases.
- **Verify:** tests in `tests/` for: tier off → internship phase `locked` and graduation after capstone; tier on → internship phase active after capstone.
- **Commit:** Use per-tier internship switch in course progress

Phase 6 — Tier 2: Virtual Internship - Backend

T-24 — Team matching

- **Files to create:** `src/lib/internships/teams.ts`, `src/app/api/internship/team/route.ts`.
- **Steps:** Pure function `formTeams(queue, windowDays = 7, min = 3, max = 4)` returning teams and leftovers; leftovers after 7 days go solo. Call it when a Tier-2 student starts, and from a daily job (if the project has a cron pattern in `src/app/api/cron/`, add a route there, protected like the others).
- **Verify:** unit tests for 1, 3, 5, 7 and 8 students.
- **Commit:** Add virtual internship team matching

T-25 — Product brief and backlog generation

- **Files to create:** `src/lib/internships/productBrief.ts`
- **Steps:** Schema `{ productName, summary, dataModel: {table, columns[][]}, stories: {id, title, acceptance: string[][]} (12-16) }`; assign stories evenly to members; solo mode gets a scaled list of 6-8.
- **Commit:** Add AI product brief and backlog for virtual internships

T-26 — Weekly checked tasks (Python + SQL)

- **Steps:** For each member and week, generate 2 tasks linked to their stories using T-10...T-12 (one `python`, one `sql`). Unlock week N tasks when sprint N opens.
- **Commit:** Add weekly checked tasks for virtual internships

T-27 — Sprints, PR links, repo, stand-ups

- **Files to create:** routes from C11 (`team/repo`, `sprint/[sprintId]/pr`, `sprint/[sprintId]/submit`, `standup`).
- **Steps:** Reuse `parseGithubLink`, `sameGithubRepo` and `probePublicUrl` (see `capstoneSprints.ts`, `publicUrlProbe.ts`). A PR URL must match `/pull/<number>` in the team repo.
- **Commit:** Add sprint, pull request and stand-up routes

T-28 — Sprint review (AI now, mentor later)

- **Files to create:** `src/lib/internships/sprintReview.ts`, `src/app/api/admin/internship/sprint/[sprintId]/review/route.ts`.
- **Steps:** When a sprint is submitted, the AI reviews it (tasks passed, PR links, stand-ups) and returns `{ decision: 'approved' | 'changes_requested', reasons[] }`. While D1 is at its default (no mentors), this decision is final for Tier 2; admins can override through the admin route. Add a config flag `MENTOR_REVIEW_REQUIRED = false` in `tiers.ts` for later.
- **Commit:** Add sprint reviews for virtual internships

T-29 — Final demo, defense and certificate for Tier 2

- **Steps:** Demo URL probed (like capstone Sprint 3). Defense uses `verifyTopicEvaluationSignature` and `isCapstoneInterviewPassed` with topic `Virtual Internship - <productName>`. Certificate rules from FR-T2-9. Add the Tier-2 screens (team card, backlog, sprint board, stand-up form) in `src/app/quests/components/internship/`.
- **Commit:** Complete the virtual internship flow

Phase 7 — Tiers 3-5 (build only after the owner decides D1 and D2)

T-30 — Opportunities (admin) — **BLOCKED** (needs owner decision D2)

Admin CRUD for `internship_opportunities`, with organisation authenticity fields from Verification Standard §6. Commit: Add `internship_opportunities admin`.

T-31 — Applications (student) — **BLOCKED** (D2)

List open opportunities for the student's tier; apply/withdraw; admin accepts/rejects.

T-32 — Weekly logs — **BLOCKED** (D2)

Weekly hours/summary/links; show totals versus the §4 minimums (8 weeks × 30 h or 16 weeks × 15 h).

T-33 — Mentor role and reviews — **BLOCKED** (needs owner decision D1)

Add a `mentor` role check (like the admin check in `requireAuth.ts`), a mentor queue page, and weekly reviews for Tiers 2-3. Then set `MENTOR_REVIEW_REQUIRED = true` for Tier 2.

T-34 — Supervisor evaluation link — **BLOCKED** (D2)

One-time token link (NFR-SEC-4), a form with the 4 rated areas, no login.

T-35 — Open-source fallback check — **BLOCKED** (D2)

GitHub API check of merged PRs (merged, author, dates, approved repo list). Cache results; respect rate limits.

T-36 — Admin verification and records — **BLOCKED** (D1, D2)

Admin verifies Tier 4-5 internships; server writes a verified `internship_records` row and issues the certificate.

T-37 — Simulated-client fallback for Tier 3 — **BLOCKED** (D1)

AI-played client chat with changing requirements; certificate says "(simulated client)".

Phase 8 — Launch Tier 1 and Tier 2

T-38 — Honest plan and preview wording

- **Files to change:** `CrashCoursePlanCards.tsx`, `CredentialPreviewModal.tsx`, `ActiveEnrollmentBanner.tsx`.
- **Steps:** Use the per-tier switch and the tier names from `tiers.ts`. Tier 1 wording: "2-Week Python Job Simulation (simulated company)". Tier 2: "4-Week Virtual Internship – Backend (team, simulated company)".
- **Commit:** Show honest internship wording per tier

T-39 — End-to-end check with a test account

- **Steps:** With the owner's permission, on a staging database: create a 1-month Python enrollment, complete it, start the Job Simulation, pass all tickets, and get the certificate. Record screenshots and any bugs. Do the same for Tier 2 with 3 test accounts.
- **Do not:** use real students' data.
- **Done when:** both flows work end to end, and the owner has reviewed the screenshots.

T-40 — Switch on Tier 1 (and later Tier 2)

- **Steps:** Only after T-39 and the owner's "yes": set `INTERNSHIP_TIER_AVAILABLE.t1_job_sim = true`. In a separate commit and PR, and only after its own owner "yes": `t2_virtual_team = true`.
- **Verify:** standard checks; update the test from T-04 so it expects internship text **only** for switched-on tiers.
- **Commit:** Switch on the Python Job Simulation
- **Done when:** merged through a PR with "Web app tests" passing.

PART E — REFERENCE

E1. Glossary

Word	Meaning
Crash enrollment	A student's purchase of a certification plan (<code>user_crash_enrollments</code>)
Tier	The internship level that belongs to a plan length (C2)
Ticket	One Tier-1 task, like a bug report at a real company
Visible tests	Checks the student can see and run
Hidden tests	Extra checks only the server runs, so students cannot just satisfy the visible ones
Reference solution	The AI's own answer, used only to prove the task is solvable
Sandbox	The server's safe Python runner (timeouts, blocked dangerous code)
Stand-up	A short daily/weekly update: done, next, blockers
Sprint	One week of team work with a goal
Mentor	A human engineer who reviews student work (D1)
Supervisor	The student's manager at a real outside company (Tiers 4-5)

E2. Handy commands

```
# Standard checks
npx tsc --noEmit && npm test && npx next build

# Rebuild the code graph after changes (installed with: pip install graphify)
graphify update src
graphify query "internship" --graph src/graphify-out/graph.json

# Find where something is used
grep -rn "INTERNSHIP_AVAILABLE" src
```

E3. Owner's short summary (non-technical)

- **Done:** the 12-month Python plan is complete, with 20-30 minute classes, checked practice tasks and tests.
- **Next:** internships, different for each plan length. 1 month gets a 2-week job simulation; 3 months a 4-week team internship; 6, 9 and 12 months get real-world internships that need mentors and partner companies.
- **Needed from you:** answers to D1 (mentors), D2 (partner companies) and D3 (names). Tiers 1 and 2 can be built without waiting.
- **Safety:** nothing is shown to students until it is built, tested and switched on with your approval.